

Aula 1: O que é back-end, Ruby e o primeiro Rails

Você sai daqui com: uma API respondendo `GET /api/v1/status`. **Gabarito:** `cd ~/capacitacao-gabarito && git checkout aula-01`

Pré-requisitos instalados em `00-preparacao.md`.

Antes de tudo: como usar esta apostila

Você não precisa entender tudo hoje. **Precisa fazer funcionar hoje:** o entendimento vem completando os buracos nas próximas semanas, e é assim mesmo que se aprende back-end.

Três coisas que vale saber antes de começar:

Você vai travar. Não é sinal de que você não serve para isso; é o trabalho. A diferença entre quem começou ontem e quem faz isso há dez anos não é travar menos: é travar melhor, e ler a mensagem de erro em vez de entrar em pânico. Toda prática desta apostila termina com uma tabela **Se der errado**, com os erros que acontecem de verdade. Ela existe exatamente para isso.

As práticas são a aula. Os textos entre elas explicam o porquê; as práticas é que fixam. Se o tempo apertar e você tiver que escolher, escolha fazer.

Perguntar é rápido. Se você travou por mais de dez minutos no mesmo ponto, pergunte. Ninguém vai achar a pergunta boba, e quem está do seu lado provavelmente está no mesmo lugar, calado.

Uma nota sobre o tom: esta apostila afirma as coisas com bastante convicção, porque explicar com mil ressalvas fica ilegível. Mas quase toda decisão aqui tem alternativa razoável. Quando você discordar de alguma, **pergunta:** essa conversa costuma valer mais que o parágrafo.

1. Os dois lados

Na capacitação de front-end você aprendeu a fazer o que o usuário vê. Back-end é o outro lado.

Front-end	Back-end
A parte visível, que o usuário toca	A parte lógica, nos bastidores
HTML, CSS, JavaScript	Ruby, Python, Java, Go, Node
Roda no navegador ou no celular	Roda num servidor, longe do usuário
Se cair, a tela quebra	Se cair, nada funciona

O back-end guarda os dados, decide quem pode ver o quê, aplica as regras de negócio, autentica usuários e conversa com outros sistemas (e-mail, pagamento, mapas).

Por que isso não pode viver no front? Porque tudo que roda no navegador o usuário consegue ler e alterar. A validação de senha no front é conveniência; a que vale é a do back.

A analogia: o salão do restaurante é o front-end. Mesa, cardápio, garçom. A cozinha é o back-end: ninguém entra, mas é lá que a comida sai. O pedido é a requisição, o prato é a resposta. Você não pede pra ver a receita; você pede o prato.

2. A rede, o mínimo necessário

O que é um servidor, afinal

Não é uma máquina especial. É um **programa esperando**: ele fica ligado, escutando numa porta, e responde ao que chegar ali. `bin/rails server` sobe esse programa na porta 3000: na sua máquina ou numa VM na nuvem, é o mesmo programa.

IP e porta

IP é o endereço da máquina (`57.156.65.151`). **Porta** é a sala dentro dela: um número de 1 a 65535. Uma máquina tem milhares de portas, e um programa diferente pode escutar em cada uma.

Porta	Quem mora ali
22	SSH
80	HTTP
443	HTTPS
5432	Postgres
3000	Rails em desenvolvimento

`127.0.0.1` (o `localhost`) significa sempre "esta máquina aqui". Na Aula 4 vamos abrir e fechar portas na mão, no firewall de um servidor Linux.

DNS

Ninguém decora `57.156.65.151`. O **DNS** é a agenda telefônica da internet: você digita `api.seemxxiii.tech` e alguém pergunta ao DNS qual é o IP.

A resposta fica em cache por um tempo: o **TTL**. É por isso que apontar um domínio para outro servidor não vale na hora. Na Aula 4 você vai criar um desses registros.

Anatomia de uma URL

```
https :// api.seemxxiii.tech : 443 /api/v1/lectures ?page=2 #topo
|         |           |         |         |         |
esquema  host      porta  caminho  query  fragmento
```

- **Esquema:** o protocolo. `http`, `https`, `ssh`, `postgres`.
- **Porta:** opcional. `http` assume 80, `https` assume 443.
- **Fragmento:** nunca vai para o servidor; é só para o navegador.

É por isso que `localhost:3000` tem os dois pontos: a porta não é a padrão.

3. HTTP

Cliente é quem pede (navegador, app, outro servidor). Servidor é quem responde. **O cliente sempre começa a conversa:** o servidor nunca liga primeiro.

Uma **requisição** tem método, caminho, cabeçalhos e corpo. Uma **resposta** tem status, cabeçalhos e corpo.

Como ela é, por dentro

```
POST /api/v1/sessions HTTP/1.1      ← método, caminho, versão
Host: api.seemxxiii.tech             |
Content-Type: application/json       | cabeçalhos
Accept: application/json             |
                                     ← linha em branco separa
{"email": "ana@ufop.br", "password": "..."} ← corpo
```

É **texto puro**. HTTP é literalmente isso trafegando num cano.

Cabeçalhos

São **metadados**: informação *sobre* a mensagem, não a mensagem. Um par `chave: valor` por linha.

Cabeçalho	Para quê
Content-Type	Em que formato vai o corpo
Accept	Em que formato eu quero a resposta
Authorization	Quem sou eu (vai ser o nosso token, na Aula 3)
Set-Cookie / Cookie	Como o navegador guarda estado
User-Agent	Que programa está chamando

Content-Type importa

O mesmo dado, dois formatos:

```
application/json      → {"email":"ana@ufop.br"}
application/x-www-form-urlencoded → email=ana%40ufop.br
```

Sem o cabeçalho, o servidor não sabe como ler o corpo. Esquecer isso no `curl` é o erro nº 1 de quem começa: vem `400` e a mensagem não ajuda em nada. Por isso todo `curl` deste material tem `-H 'Content-Type: application/json'`.

Os métodos

Método	Significa
GET	Me dá isso. Não muda nada.
POST	Cria isso.
PUT / PATCH	Altera isso.
DELETE	Apaga isso.

O método é uma promessa. Um `GET` que apaga registro é bug, não criatividade.

Os status

Faixa	Significa	Exemplos
2xx	Deu certo	<code>200</code> ok, <code>201</code> criado
3xx	Foi pra outro lugar	<code>301</code> , <code>302</code>
4xx	Você errou	<code>400</code> pedido mal feito, <code>401</code> não autenticado, <code>403</code> sem permissão, <code>404</code> não existe, <code>422</code> dado inválido
5xx	Nós erramos	<code>500</code> bug nosso

`401` é "quem é você?". `403` é "sei quem você é, e você não pode". Vamos usar os dois na Aula 3.

HTTP não tem memória

Cada requisição começa do zero. O servidor esquece tudo entre uma e outra.

Guarde essa frase. Ela é o problema inteiro da Aula 3: se o servidor esquece tudo, como ele sabe que você já fez login?

Prática 1: HTTP com as próprias mãos

É a primeira vez que você fala com um servidor sem navegador no meio. Faça comando por comando, e **leia a saída** de cada um antes de passar para o próximo.

a) A requisição mais simples que existe

```
curl -i https://api.seemxxiii.tech/api/v1/status
```

```
HTTP/2 200
content-type: application/json; charset=utf-8

{"status":"ok","service":"seem-backend"}
```

O `-i` manda o `curl` mostrar os **cabeçalhos** junto com o corpo. Isso é uma API real, no ar: é exatamente o que você vai construir.

b) Veja a requisição inteira, dos dois lados

```
curl -v https://api.seemxxiii.tech/api/v1/status
```

Linhas com `>` são o que **você mandou**; com `<`, o que o **servidor respondeu**. Ache no meio:

- a linha `> GET /api/v1/status HTTP/2`: o método e o caminho
- o cabeçalho `> user-agent: curl/...`: quem você disse que era
- a linha `< HTTP/2 200`: o status

c) Provoque um erro de propósito

Antes de rodar, responda para você: o servidor vai responder alguma coisa, ou a conexão vai falhar? Responda de verdade, mentalmente, e só depois rode. Prever antes de observar é o que transforma "vi acontecer" em "entendi".

```
curl -i https://api.seemxxiii.tech/api/v1/rota-que-nao-existe
```

Veio `404`. Repare que o servidor **respondeu**: 404 não é "deu erro de conexão", é uma resposta perfeitamente bem-sucedida dizendo "esse recurso não existe".

d) Só os cabeçalhos

```
curl -I https://api.seemxxiii.tech/api/v1/status
```

`-I` faz um `HEAD`: pede só os cabeçalhos, sem o corpo. É como um monitoramento checa se o site está de pé sem baixar a página inteira.

e) Avançado: mande um POST e veja o `Content-Type` importar

```
curl -i -X POST https://api.seemxxiii.tech/api/v1/status \
-H "Content-Type: application/json" \
-d '{"oi": true}'
```

A rota de status só aceita `GET`, então você vai levar um `404` ou `405`. **O ponto não é o sucesso** é você ter montado uma requisição com método, cabeçalho e corpo na mão, que é o que o Insomnia faz por baixo.

Confere: você consegue apontar, na saída do `-v`, onde termina a sua requisição e onde começa a resposta. E consegue dizer o que significa cada um dos três números que viu: 200, 404 e o do POST.

Se der errado

Erro	Causa	Saída
<code>curl: command not found</code>	o <code>curl</code> não está instalado	<code>sudo apt install curl</code> (ou <code>brew install curl</code>)
<code>Could not resolve host</code>	sem internet, ou o nome está errado	teste <code>curl -i https://example.com</code>
<code>Connection refused</code>	o servidor não está no ar	avise no grupo: pode ser a API que caiu, e não você
<code>curl: (60) SSL certificate problem</code>	relógio da máquina errado, ou proxy da rede	confira a data do sistema; na Aula 4 você vai entender esse erro por dentro
a saída sai toda numa linha só, ilegível	faltou o <code>-i</code> , ou o JSON não tem quebra	acrescente <code>\ python3 -m json.tool</code> no fim

4. API REST

API é a porta de entrada do sistema para outros programas. **REST** é um estilo de organizar essa porta: cada coisa do sistema é um **recurso**, com um endereço.

```
/api/v1/lectures    → as palestras
/api/v1/lectures/7  → a palestra 7
```

O que você faz com o recurso é o **verbo**, não o endereço:

GET	/api/v1/lectures	lista
POST	/api/v1/lectures	cria
GET	/api/v1/lectures/7	mostra uma
DELETE	/api/v1/lectures/7	apaga

`/criarPalestra` · `POST /lectures`

O `v1` no caminho é a versão. Quando a API mudar de forma incompatível, nasce uma `/v2` e a `/v1` continua funcionando, porque o app na loja do celular demora dias para atualizar.

5. Ruby, partindo do Python

Ruby foi criado por **Yukihiro Matsumoto (Matz)** em 1995, no Japão, com um objetivo declarado: *fazer o programador feliz*. É interpretada, dinâmica e orientada a objetos, como Python.

Esta seção é a **referência de sintaxe da capacitação**. Não é a linguagem inteira: é exatamente o que aparece nas quatro aulas, na ordem em que aparece, e cada bloco diz **onde no projeto você vai encontrar aquilo**. A tabela lado a lado mais completa está em `ruby-para-pythonistas.md`.

Abra um `irb` numa aba do terminal e vá colando. Ler sintaxe não fixa; digitar fixa.

5.1 O primeiro parágrafo

<pre># Python def saudacao(nome, formal=False): if formal: return f"Prezado {nome}" return f"Oi, {nome}"</pre>	<pre># Ruby def saudacao(nome, formal: false) return "Prezado #{nome}" if formal "Oi, #{nome}" end</pre>
--	--

Quatro coisas para reparar, e são elas que fazem Ruby parecer estranho no primeiro dia:

1. `end` fecha o bloco, em vez da indentação.
2. `if` **no fim da linha**: é o *modificador*, e é muito usado.
3. **A última linha avaliada é o retorno**. `return` só se você quiser sair antes.
4. `#{}` interpola, como o `f"..."` do Python.

Não há ponto e vírgula, e parênteses na chamada são opcionais: `puts "oi"` e `puts("oi")` são a mesma coisa. O projeto usa parênteses quando há argumento e omite quando não há.

5.2 Variáveis e o que é "falso"

<code>nome = "Ana"</code>	<code># variável local</code>
<code>@nome = "Ana"</code>	<code># variável de instância (do objeto)</code>
<code>NOME = "Ana"</code>	<code># constante (maiúscula)</code>

O `@` é o `self` do Python, só que sem precisar declarar no `__init__`. Dentro de uma classe, `@email` é o atributo daquele objeto.

A pegadinha número um de quem vem do Python:

```
if 0      then puts "entrou" end  # ENTRA. 0 é verdadeiro em Ruby.
if ""     then puts "entrou" end  # ENTRA. string vazia é verdadeira.
if []     then puts "entrou" end  # ENTRA. array vazio é verdadeiro.
if nil    then puts "entrou" end  # não entra
if false  then puts "entrou" end  # não entra
```

Só `nil` e `false` são falsos. Todo o resto é verdadeiro. Em Python, `0`, `""`, `[]` e `{}` são todos falsos: aqui, não.

É por isso que no projeto você vai ver `if params[:email].present?` e não `if params[:email]`: a string vazia passaria pela segunda.

	Ruby	Python
ausência de valor	<code>nil</code>	<code>None</code>
vazio conta como falso?	não	sim
testar "tem conteúdo?"	<code>.present?</code> (Rails)	<code>if x</code>
testar "está vazio?"	<code>.blank?</code> (Rails)	<code>if not x</code>

5.3 Tudo é objeto

Em Python você chama `len(x)`. Em Ruby, `x.length`. **Não existe função solta**: é sempre método de alguém. Até número é objeto.

```
3.times { puts "oi" }
-5.abs      # 5
"ana".capitalize # "Ana"
nil.to_a    # []
1.class     # Integer
```

Isso muda como você lê o código: quando vir `user.email.strip.downcase`, leia da esquerda para a direita, cada método operando no resultado do anterior.

5.4 Strings e símbolos

```
'texto simples'      # aspas simples: não interpola
"olá, #{nome}"       # aspas duplas: interpola
"linha\n"            # aspas duplas: entende \n
'linha\n'             # aspas simples: dois caracteres, \ e n
```

Use aspas simples quando não há nada para interpolar: é a convenção, e o RuboCop cobra.

Símbolo é um texto que serve de **etiqueta**, não de conteúdo:


```
:aluno :email :admin
```

O mesmo símbolo é sempre o mesmo objeto na memória; duas strings iguais são dois objetos. Não existe em Python. Lá você usaria uma string.

Regra prática: conteúdo que o usuário lê → string. Nome de coisa no código → símbolo.

No projeto, símbolo aparece em chave de hash (`params[:email]`), em nome de validação (`validates :email, presence: true`) e em nome de callback (`before_action :autenticar!`).

5.5 Coleções

```
# Array – a lista do Python
nomes = ["ana", "bia", "caio"]
nomes[0]      # "ana"
nomes[-1]     # "caio"
nomes << "davi" # append
nomes.first   # "ana"
nomes.size    # 4

# Hash – o dict do Python
user = { nome: "Ana", role: :admin }
user[:nome]   # "Ana"
user[:idade]  # nil (NÃO estoura, diferente do dict do Python)
user.fetch(:idade) # aí sim: KeyError
user.fetch(:idade, 0) # 0
```

Repare em `{ nome: "Ana" }`: é o atalho para `{ :nome => "Ana" }`. **A chave é um símbolo**, não uma string, e `user["nome"]` devolveria `nil`. Esse é o erro mais comum da primeira semana.

`fetch` é importante: ele **falha alto** quando a chave não existe. É por isso que o `config/deploy.yml` da Aula 4 usa `ENV.fetch("SERVER_IP")`: se a variável não estiver definida, você descobre na hora, e não três passos depois.

5.6 Condicionais

```
if idade >= 18
  puts "maior"
elsif idade >= 16
  puts "vota"
else
  puts "menor"
end

unless ativo?      # o "if not" do Ruby
  puts "inativo"
end

puts "maior" if idade >= 18      # modificador: cabe numa linha
puts "inativo" unless ativo?

status = idade >= 18 ? "maior" : "menor"    # ternário, igual ao C
```

E o `case`, que aceita mais coisa que o do Python:

```
case status
when "ok"          then 200
when "not_found"   then 404
when 400..499      then "erro do cliente"    # aceita intervalo
else               500
end
```

5.7 Blocos

Um bloco é um pedaço de código que você entrega para um método executar. É o `lambda` do Python, mas usado o tempo todo, e é a construção mais característica da linguagem.

```
usuarios.each do |u|          # for u in usuarios
  puts u.nome
end

usuarios.map { |u| u.nome }    # [u.nome for u in usuarios]
usuarios.select { |u| u.ativo? } # [u for u in usuarios if u.ativo]
usuarios.reject { |u| u.ativo? } # o inverso
usuarios.find { |u| u.admin? }  # o primeiro que casar, ou nil
usuarios.any? { |u| u.admin? }  # true/false
usuarios.count { |u| u.admin? } # quantos
```

Chaves quando cabe numa linha, `do ... end` quando não cabe. É convenção, e o RuboCop cobra.

O que está entre `{ | | }` são os parâmetros do bloco. Com dois:

```
{ a: 1, b: 2 }.each { |chave, valor| puts "#{chave}=# {valor}" }
```

Blocos também servem para **delimitar um trecho**, e é assim que o projeto usa transação:

```
User.transaction do
  user.save!
  LoginEvent.create!(user: user)
end
```

Tudo dentro do bloco acontece, ou nada acontece.

5.8 Métodos e argumentos nomeados

```
def somar(a, b)          # posicionais
  a + b
end

def login(email:, password:, client: "mobile") # nomeados
  # ...
end

login(email: "a@b.c", password: "x")
```

Os dois-pontos **depois** do nome (`email:`) tornam obrigatório nomear na chamada. Sem valor padrão, o argumento é obrigatório; com valor padrão (`client: "mobile"`), é opcional.

O projeto usa nomeados em todo service, de propósito: `Users::Register.call(email:, password:)` se lê sozinho, e `Users::Register.call("a@b.c", "x")` não.

O atalho do Ruby 3.1: quando a variável tem o mesmo nome da chave, você omite o valor.

```
user = User.first
Result.new(success?: true, user:) # é o mesmo que user: user
```

Isso aparece em todo service do projeto. **Não é erro de digitação.**

5.9 ? e !

Sufixo	Significa	Exemplo
?	Devolve verdadeiro ou falso	<code>user.admin?</code> , <code>email.blank?</code>
!	A versão perigosa: estoura erro em vez de devolver <code>false</code>	<code>user.save!</code>

É convenção, não regra da linguagem. Mas todo mundo segue, e o Rails também:

```
user.save # false se as validações falharem – você tem que checar
user.save! # levanta ActiveRecord::RecordInvalid
```

No projeto, dentro de uma transação usamos sempre a versão com `!`: se falhar no meio, tem que estourar para a transação desfazer tudo. Um `save` silencioso ali deixaria o banco pela metade.

5.10 Lidando com `nil`

Três atalhos que aparecem o tempo todo, e que economizam muito `if`:

```
user&.email      # se user for nil, devolve nil em vez de estourar
                  # (é o "safe navigation")

@lista ||= []    # atribui SÓ se estiver nil ou false
                  # ("ou-igual": o padrão de memoização do Ruby)

nome = params[:nome] || "anônimo"  # valor padrão
```

Sem o `&.`, `user.email` com `user` valendo `nil` levanta `NoMethodError: undefined method 'email' for nil`. **Esse é o erro que você mais vai ver**, e quando vir, a pergunta certa é "quem aqui virou `nil`?".

E dois métodos que o Rails acrescenta e o Ruby puro não tem:

```
".blank?      # true    - nil, "", "   ", [] e {} são todos "blank"
".present?    # false
"ana".present? # true
```

5.11 Classes

```
class Usuario
  attr_reader :email      # cria o método usuario.email
  attr_accessor :nome      # cria o .nome e o .nome=

  def initialize(email:, nome: nil)
    @email = email
    @nome = nome
  end

  def admin?
    @email.end_with?("@automic.com")
  end

  private                 # daqui para baixo, só de dentro do objeto

  def normalizar
    @email = @email.strip.downcase
  end
end

u = Usuario.new(email: "ANA@x.com ")
u.email      # "ANA@x.com "
u.admin?     # false
```

`initialize` é o `__init__`. `attr_reader` gera o getter, e evita escrever um método de uma linha para cada atributo. E `private` vale **da linha em diante**, não por método.

Método de classe (o `@staticmethod` do Python) leva `self.`:

```
class Users::Register
  def self.call(email:, password:)
    new(email:, password:).call
  end
end
```

Todo service do projeto tem exatamente esse formato: um `self.call` que instancia e chama.

5.12 Struct

Quando você só quer agrupar valores, sem comportamento:

```
Result = Struct.new(:success?, :user, :errors, keyword_init: true)

r = Result.new(success?: true, user: ana, errors: [])
r.success?  # true
r.user      # ana
```

É o `namedtuple` / `dataclass` do Python. **Todo service deste projeto devolve um desses:** é o que permite ao controller escrever `if resultado.success?` sem saber nada de dentro do service.

5.13 Módulos e mixins

Ruby não tem herança múltipla. Tem **módulo**: um saco de métodos que você inclui numa classe.

```
module EmailVerifiable
  def gerar_codigo_de_verificacao
    # ...
  end
end

class User < ApplicationRecord
  include EmailVerifiable
  include PasswordResettable
end
```

Vamos escrever esses dois na Aula 2. O nome deles no mundo Rails é **concern**, e o motivo de existirem é manter o `User` legível: sem eles, o model teria 200 linhas misturando três assuntos.

Módulo também serve de **namespace**, e é assim que o projeto organiza as pastas:

```
module Users
  class Register          # a classe é Users::Register
  end                    # e o arquivo é app/services/users/register.rb
end
```

5.14 Exceções

Python	Ruby
<code>try</code>	<code>begin</code>
<code>except</code>	<code>rescue</code>
<code>finally</code>	<code>ensure</code>
<code>raise</code>	<code>raise</code>
base para capturar	<code>StandardError</code> (não <code>Exception</code>)

```
def call
  arriscado
rescue StandardError => e
  Rails.error.report(e)
  nil
end
```

Dentro de um método o `begin` é implícito: dá para escrever só o `rescue` no fim. Criando o seu tipo de erro:

```
class InvalidToken < StandardError; end

raise InvalidToken if token_ruim?
```

Herde sempre de `StandardError`, nunca de `Exception`: capturar `Exception` pega até `Ctrl+C`.

5.15 Dependências

Python	Ruby
<code>pip install</code>	<code>gem install</code>
<code>requirements.txt</code>	<code>Gemfile</code>
<code>venv</code> por projeto	<code>bundler</code> resolve tudo
<code>pip freeze</code> para travar	<code>Gemfile.lock</code> , gerado sozinho
<code>python script.py</code>	<code>ruby script.rb</code>
<code>python -m pytest</code>	<code>bundle exec rspec</code> / <code>bin/rails test</code>

O `Gemfile.lock` é o `requirements.txt` travado: só que automático e **sempre versionado**. Ele é a garantia de que a sua máquina e o servidor rodam exatamente as mesmas versões.

E o `bundle exec` na frente do comando significa "rode usando as versões do `Gemfile.lock`", não as que estiverem soltas na máquina".

5.16 O mapa: onde cada coisa aparece

Sintaxe	Onde você vai encontrar
símbolo, hash	<code>params[:email]</code> , validações (Aula 2 e 3)
bloco	<code>each</code> , <code>map</code> , <code>User.transaction do</code> (Aula 2)
<code>?</code> e <code>!</code>	<code>user.save!</code> , <code>valid?</code> (Aula 2)
<code>&.</code> e <code>\ \ =</code>	tratamento de <code>nil</code> nos controllers (Aula 3)
argumento nomeado + atalho 3.1	todo service (Aula 3)
<code>Struct</code>	o retorno de todo service (Aula 3)
módulo / <code>include</code>	os concerns do <code>User</code> (Aula 2)
namespace com módulo	<code>Api::V1::SessionsController</code> (Aula 3)
<code>rescue</code>	tratamento de erro da API (Aula 3)
<code>ENV.fetch</code>	<code>config/deploy.yml</code> (Aula 4)

Prática 2: Ruby no `irb`

Não pule esta: é a única vez na capacitação em que você mexe em Ruby sem Rails no caminho, e é aqui que a sintaxe entra.

Abra o console interativo:

```
irb
```

a) Tudo é objeto

```
3.class      # Integer
"oi".class   # String
:oi.class    # Symbol
nil.class    # NilClass
3.times { |i| puts i }
```

b) A pegadinha do "falso"

```
puts "entrou" if 0      # entra! 0 é verdadeiro
puts "entrou" if ""     # entra!
puts "entrou" if nil    # não entra
```

Se você veio do Python, pare cinco segundos aqui.

c) Hash e símbolo

```

usuario = { nome: "Ana", role: :admin }
usuario[:nome]      # "Ana"
usuario["nome"]     # nil  <- por quê?
usuario[:idade]     # nil
usuario.fetch(:idade) # KeyError

```

Responda para você mesmo por que `usuario["nome"]` deu `nil`.

d) Blocos

```

[1, 2, 3].map { |n| n * n }
[1, 2, 3].select { |n| n.odd? }
%w[ana bia caio].each { |n| puts n.capitalize }

```

`%w[...]` é o atalho para array de strings: aparece bastante em código Rails.

e) Nil, e como não apanhar dele

```

user = nil
user.email      # NoMethodError – leia a mensagem inteira
user&.email     # nil, sem estourar
nome = user&.email || "anônimo"

```

f) Um método, com nomeados e retorno implícito

```

def saudacao(nome:, formal: false)
  return "Prezado #{nome}" if formal
  "Oi, #{nome}"
end

saudacao(nome: "Ana")
saudacao(nome: "Ana", formal: true)
saudacao("Ana")      # ArgumentError – leia a mensagem

```

g) Um `Struct`, que é o que todo service vai devolver

```

Result = Struct.new(:success?, :user, :errors, keyword_init: true)
r = Result.new(success?: true, user: "ana", errors: [])
r.success?
r.errors

```

h) Avançado: escreva um módulo e inclua numa classe

Faça isto sem olhar a seção 5.13, e depois confira.


```

module Saudavel
  def cumprimentar = "Oi, #{nome}"
end

class Pessoa
  include Saudavel
  attr_reader :nome
  def initialize(nome) = @nome = nome
end

Pessoa.new("Ana").cumprimentar

```

O `def metodo = expressão` é o *endless method*, do Ruby 3. Serve para método de uma linha só.

Confere: você conseguiu explicar, em voz alta, por que `usuario["nome"]` deu `nil` e por que `saudacao("Ana")` deu `ArgumentError`. Se conseguiu, a sintaxe entrou.

Se der errado

Erro	Causa	Saída
<code>irb: command not found</code>	o <code>mise</code> não está ativo no shell	<code>exec bash</code> e confira com <code>which ruby</code>
<code>NameError: undefined local variable</code>	você digitou o nome errado, ou perdeu o <code>@</code>	releia a linha; Ruby não avisa antes de rodar
<code>NoMethodError: undefined method 'x' for nil</code>	alguma coisa virou <code>nil</code> antes	a pergunta certa é "quem virou <code>nil</code> ?", não "o que é esse método"
<code>ArgumentError: missing keyword: :nome</code>	o método pede argumento nomeado e você passou posicional	<code>saudacao(nome: "Ana")</code>
<code>syntax error, unexpected end-of-input</code>	faltou um <code>end</code>	conte os <code>def / do / if</code> abertos

6. Rails

Framework web em Ruby, criado por **David Heinemeier Hansson** em 2004: extraído do Basecamp, um produto real. Traz tudo junto: rotas, banco, e-mail, filas, testes, deploy. Estamos na versão 8.

As duas leis

Convenção sobre configuração. Você não configura o óbvio, você segue o combinado. Classe `User`? Então a tabela é `users`, o arquivo é `user.rb`, a chave primária é `id`. Ninguém precisa dizer. Seguindo a convenção você escreve quase nada; brigando com ela, o dobro.

DRY: *Don't Repeat Yourself*. Cada regra existe num lugar só.

E uma atitude: **omakase**, "deixa comigo, chef". O Rails já escolheu as ferramentas por você. Você pode trocar, mas só troque quando tiver um motivo real. Isso é liberdade: sobra tempo pro problema de verdade.

Rails × Django

Django	Rails
<code>models.py</code>	<code>app/models/</code> , um arquivo por classe
<code>makemigrations</code> / <code>migrate</code>	<code>rails g migration</code> / <code>db:migrate</code>
<code>urls.py</code>	<code>config/routes.rb</code>
<code>views.py</code>	<code>app/controllers/</code>
Django ORM	ActiveRecord
<code>manage.py</code>	<code>bin/rails</code>

Quem vem de Flask ou FastAPI vai sentir mais diferença: lá você monta cada peça, aqui elas já vêm encaixadas.

MVC

- **Model:** os dados e as regras. Conversa com o banco.
- **View:** a tela. Na nossa API, é o JSON.
- **Controller:** recebe a requisição e decide o que fazer.
- **Rota:** o mapa. Este endereço vai para aquele controller.

O caminho é sempre: **rota** → **controller** → **model** → **resposta**.

Zeitwerk: o nome diz o caminho

Você nunca escreve `require` no Rails. Por quê?

O **Zeitwerk** carrega a classe no instante em que você a menciona, e descobre o arquivo pelo **nome da classe**:

```
Api::V1::StatusController → app/controllers/api/v1/status_controller.rb
```

Errou o caminho, a classe simplesmente não existe. Não é questão de estilo: é o mecanismo que faz "convenção sobre configuração" funcionar de verdade.

Os três ambientes

Ambiente	Onde	Como se comporta
<code>development</code>	sua máquina	recarrega o código a cada requisição, log verboso, erro na tela
<code>test</code>	os testes	banco separado, limpo a cada teste
<code>production</code>	o servidor	código congelado, log enxuto, erro genérico

Cada um tem um arquivo em `config/environments/`. `Rails.env` diz onde você está.

É assim que o e-mail abre no navegador em desenvolvimento e sai por SMTP em produção: mesmo código, ambientes diferentes.

7. Mão na massa

Criar o projeto

```
rails new automic_auth_api --api -d postgresql \
  --skip-action-mailbox --skip-action-text --skip-active-storage \
  --skip-jbuilder --skip-action-cable
cd automic_auth_api
```

- `--api`: sem HTML, sem CSS, sem sessão de navegador. Só JSON.
- `-d postgresql`: o mesmo banco que usamos em produção.
- Os `--skip` tiram peças que não vamos usar. Cada peça a menos é uma peça a menos para dar problema.

Subir o banco

Crie o `compose.yaml` (ou copie o deste repositório) e rode:

```
docker compose up -d
docker compose ps          # tem que aparecer "healthy"
```

Se der `address already in use`, você já tem um Postgres na máquina ocupando a 5432. Rode `DB_PORT=5433 docker compose up -d` e exporte `DB_PORT=5433` no shell.

Preparar e subir a aplicação

```
bin/rails db:prepare
bin/rails server
```

Abra `http://localhost:3000/up`. Verde é a aplicação de pé.

O tour das pastas

Pasta	O que tem
<code>app/</code>	O seu código. É onde você passa 90% do tempo.
<code>config/</code>	Rotas, banco, ambientes, credenciais
<code>db/</code>	Migrations e o retrato atual do banco (<code>schema.rb</code>)
<code>test/</code>	Os testes
<code>bin/</code>	Os comandos: <code>rails</code> , <code>setup</code> , <code>dev</code>
<code>Gemfile</code>	As dependências

Os comandos do dia a dia

```
bin/rails server      # sobe a aplicação
bin/rails console     # um irb com o seu projeto carregado dentro
bin/rails routes      # todas as rotas que existem
bin/rails generate    # cria arquivos seguindo a convenção
bin/rails db:migrate  # aplica as mudanças de banco
bin/rails test        # roda os testes
```

O **console** é a ferramenta mais subestimada do Rails: dá para criar usuário, rodar query e testar método sem escrever um arquivo.

8. A primeira rota

`config/routes.rb`:

```
Rails.application.routes.draw do
  get "up" => "rails/health#show", as: :rails_health_check

  namespace :api do
    namespace :v1 do
      get "status", to: "status#show"
    end
  end
end
```

`app/controllers/api/v1/status_controller.rb`:

```

module Api
  module V1
    class StatusController < ApplicationController
      def show
        render json: {
          status: "ok",
          service: "atomic-auth-api",
          environment: Rails.env
        }
      end
    end
  end
end

```

Repare na convenção: `namespace :api` + `namespace :v1` + `status#show` implica exatamente o arquivo `app/controllers/api/v1/status_controller.rb`, classe `Api::V1::StatusController`, método `show`. Ninguém configurou isso.

Teste

`test/controllers/api/v1/status_controller_test.rb`:

```

require "test_helper"

class Api::V1::StatusControllerTest < ActionDispatch::IntegrationTest
  test "responde ok" do
    get "/api/v1/status"

    assert_response :ok
    assert_equal "ok", response.parsed_body["status"]
  end
end

```

```
bin/rails test
```

As práticas da aula

Sete práticas, em ordem crescente. As duas primeiras você já fez no meio da aula:

#	O que	Onde
1	HTTP com as próprias mãos	seção 3
2	Ruby no <code>irb</code>	seção 5
3	Crie o seu projeto	aqui
4	Suba o banco e a aplicação	aqui
5	A sua primeira rota	aqui
6	O teste	aqui
7	Commit e GitHub	aqui

Onde você trabalha: no **seu** projeto, que você cria na Prática 3. O repositório da capacitação é o **gabarito**: abra para consultar, não para escrever.

Prática 3: Crie o seu projeto

```
cd ~
rails new automic_auth_api --api -d postgresql \
  --skip-action-mailbox --skip-action-text --skip-active-storage \
  --skip-jbuilder --skip-action-cable
cd automic_auth_api
```

Cada flag tira uma parte do Rails que este projeto não usa. `--api` é a mais importante: gera um Rails sem views e sem os middlewares de navegador.

Confere:

```
ls # tem app/, config/, db/, test/
cat Gemfile | head -5 # a linha do rails, com a versão
bin/rails -v # Rails 8.1.x
```

Se der errado

Erro	Causa	Saída
<code>rails: command not found</code>	o <code>mise</code> não está ativo neste shell	<code>exec bash</code> , depois <code>which ruby</code> : tem que apontar para dentro de <code>~/.local/share/mise</code>
<code>Could not find gem 'rails'</code>	Ruby do sistema, não o do <code>mise</code>	<code>mise use -g ruby@3.4.9</code> e <code>gem install rails</code>
<code>You don't have write permissions</code>	you está usando o Ruby do sistema com <code>sudo</code>	nunca use <code>sudo gem install</code> ; conserte o <code>mise</code>
a pasta <code>automic_auth_api</code> já existe	you rodou duas vezes	<code>rm -rf ~/automic_auth_api</code> e refaça, ou escolha outro nome
trava em <code>Bundle complete</code> por minutos	está compilando gem nativa	é normal na primeira vez; espere

Prática 4: Suba o banco e a aplicação

É a prática em que mais gente trava, e quase sempre é porta ocupada.

Crie um arquivo `compose.yaml` na raiz do projeto:

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: automic
      POSTGRES_PASSWORD: automic
      POSTGRES_DB: automic_auth_api_development
    ports:
      - "${DB_PORT:-5432}:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U automic"]
      interval: 5s
      retries: 10

volumes:
  pgdata:
```

E em `config/database.yml`, dentro do bloco `default: &default`, acrescente:

```
host: <%= ENV.fetch("DB_HOST", "localhost") %>
port: <%= ENV.fetch("DB_PORT", 5432) %>
username: <%= ENV.fetch("DB_USER", "automic") %>
password: <%= ENV.fetch("DB_PASSWORD", "automic") %>
```

```
docker compose up -d
docker compose ps          # tem que aparecer "healthy"

bin/rails db:prepare
bin/rails server
```

Confere: abra <http://localhost:3000/up>. Verde é a aplicação de pé, falando com o banco.

Se der errado

Esta é, de longe, a prática em que mais gente trava: em toda turma, em qualquer material. Os dois erros de sempre são porta ocupada e permissão do Docker, e **nenhum dos dois é sobre programar**. São detalhes de ambiente que todo mundo enfrenta uma vez na vida e depois nunca mais. Achou o seu na tabela? Resolve em dois minutos.

Erro	Causa	Saída
<code>address already in use</code> na 5432	you already have a PostgreSQL on the machine	<code>DB_PORT=5433 docker compose up -d</code> e <code>export DB_PORT=5433</code> no mesmo shell do <code>rails</code>
<code>permission denied</code> no <code>docker</code>	you are not in the <code>docker</code> group	<code>sudo usermod -aG docker \$USER</code> , e saia e entre de novo (reabrir o terminal não basta)
<code>docker: command not found</code> no WSL2	lack of Docker Desktop integration	Docker Desktop → Settings → Resources → WSL Integration → enable the distro
<code>PG::ConnectionBad: could not connect</code>	the container has not started, or the port is not open	<code>docker compose ps</code> : the status must be <code>healthy</code> , not <code>starting</code>
<code>PG::ConnectionBad: password authentication failed</code>	the <code>database.yml</code> does not match the <code>compose.yml</code>	user and password must be <code>automatic</code> in both
<code>ActiveRecord::NoDatabaseError</code>	the database was not created	<code>bin/rails db:prepare</code> (it creates and migrates)
the page <code>/up</code> is red	the Rails app started but did not reach the database	it's the same problem as above; read the log of <code>rails server</code>
<code>Address already in use - bind(2) for 127.0.0.1:3000</code>	there is already a Rails running	check with <code>lsof -i :3000</code> and kill it, or <code>bin/rails server -p 3001</code>

Prática 5: A sua primeira rota

O coração da aula.

Em `config/routes.rb`, dentro do `Rails.application.routes.draw do`:

```
namespace :api do
  namespace :v1 do
    get "status", to: "status#show"
  end
end
```

Crie o arquivo `app/controllers/api/v1/status_controller.rb`: o caminho tem que ser exatamente esse, é o Zeitwerk que exige:

```
module Api
  module V1
    class StatusController < ApplicationController
      def show
        render json: {
          status: "ok",
          service: "automic-auth-api",
          environment: Rails.env
        }
      end
    end
  end
end
```

Confere:

```
bin/rails routes -g api          # a sua rota tem que aparecer na lista
curl -i localhost:3000/api/v1/status
```

Tem que vir `200` e o JSON. Depois monte a mesma requisição no Insomnia e confira o status ali também: é a ferramenta que você vai usar nas Aulas 3 e 4.

Se der errado

O erro campeão aqui é `uninitialized constant`. Ele assusta porque parece dizer que a sua classe não existe, e o que ele está dizendo, na verdade, é *"procurei no caminho que o nome promete e não achei"*. É o Zeitwerk da seção 6, e a correção é sempre no nome do arquivo ou da pasta.

Erro	Causa	Saída
<code>uninitialized constant Api::V1::StatusController</code>	o arquivo está no caminho errado	o caminho tem que ser <code>app/controllers/api/v1/status_controller.rb</code> , tudo minúsculo, com <code>_controller</code>
<code>uninitialized constant Api::V1</code>	você criou a classe como <code>class Api::V1::StatusController</code> sem os módulos abertos, ou faltou uma pasta	use os três <code>module</code> / <code>class</code> exemplo
<code>No route matches [GET] "/api/v1/status"</code>	a rota não entrou	<code>bin/rails routes -g api;</code> <code>namespace</code> está fora do <code>draw</code>
a rota lista como <code>/api/v1/status</code> mas dá 404	você está batendo em outra porta	confira em que porta o <code>rails</code> está rodando
<code>ActionController::RoutingError ... status#show</code>	o método <code>show</code> não existe no controller	o nome do método tem que ser <code>show</code> <code>to: "status#show"</code>
<code>NameError: undefined local variable 'Rails'</code>	erro de digitação (<code>rails.env</code>)	Ruby é sensível a maiúsculas
mudou o código e nada mudou na resposta	você está em <code>production</code> , que faz cache	em desenvolvimento o Rails não faz cache, confira <code>Rails.env</code>

Prática 6: O teste

Um teste que você escreve hoje é o que vai te avisar, na Aula 4, que o deploy quebrou alguma coisa.

Crie `test/controllers/api/v1/status_controller_test.rb`:

```
require "test_helper"

class Api::V1::StatusControllerTest < ActionDispatch::IntegrationTest
  test "responde ok" do
    get "/api/v1/status"

    assert_response :ok
    assert_equal "ok", response.parsed_body["status"]
  end
end
```

```
bin/rails test
```

Confere: sai `1 runs, 2 assertions, 0 failures`. Agora **quebre de propósito:** troque `"ok"` por `"OK"` no teste, rode de novo, e leia a mensagem de falha inteira. Depois desfaça.

Ver o teste falhar é o que prova que ele está realmente testando alguma coisa.

Se der errado

Erro	Causa	Saída
<code>ActiveRecord::PendingMigrationError</code>	o banco de teste está atrasado	<code>bin/rails db:test:prepare</code>
<code>PG::ConnectionBad</code> no teste	o container do banco não está de pé	<code>docker compose up -d</code>
<code>NoMethodError: parsed_body</code>	versão antiga do Rails	confira <code>bin/rails -v</code> ; este material é para o Rails 8
<code>0 runs, 0 assertions</code>	o arquivo não foi encontrado	o nome tem que terminar em <code>_test.rb</code> e estar dentro de <code>test/</code>
<code>NameError: uninitialized constant ...ControllerTest</code>	o nome da classe não bate com o do arquivo	<code>status_controller_test.rb</code> → <code>StatusControllerTest</code>

Prática 7: Commit e GitHub

Não é opcional: na Aula 4 o CI roda no **seu** repositório e a imagem Docker vai para a **sua** conta.

```
git add -A
git commit -m "Rota de status"
gh repo create automic_auth_api --private --source=. --push
```

Sem o `gh` instalado, crie o repositório pelo site e depois:

```
git remote add origin git@github.com:SEU-USUARIO/automic_auth_api.git
git push -u origin main
```

Confere:

```
gh repo view --web      # abre o seu repositório no navegador
git log --oneline       # os seus commits estão lá
```

Se der errado

Erro	Causa	Saída
<code>gh: command not found</code>	o <code>gh</code> não está instalado	<code>sudo apt install gh</code> / <code>brew install gh</code>
<code>gh auth status</code> diz que não está logado	falta autenticar	<code>gh auth login</code> → GitHub.com → HTTPS → pelo navegador
<code>Permission denied (publickey)</code> no push por SSH	a sua chave não está no GitHub	<code>gh ssh-key add ~/.ssh/id_ed25519.pub</code>
<code>remote origin already exists</code>	you rodou o <code>remote add</code> duas vezes	<code>git remote set-url origin <url></code>
<code>src refspec main does not match any</code>	you ainda não commitou nada	<code>git add -A && git commit -m "..."</code> primeiro
<code>Updates were rejected</code>	o repositório remoto tem commits que você não tem	<code>git pull --rebase origin main</code> e empurre de novo

Bônus, se sobrou tempo

- Faça a rota devolver também `RUBY_VERSION` e `Rails.version`.
- Acrescente uma asserção no teste para os dois campos novos.
- Faça `curl -i localhost:3000/api/v1/status` e confira que o `content-type` é `application/json`. Por que ele é isso e não `text/html`?

Travou em qualquer prática? Abra o mesmo arquivo no gabarito, entenda o que está diferente, e conserte o seu:

```
cd ~/capacitacao-gabarito && git checkout aula-01
cat app/controllers/api/v1/status_controller.rb
```

Recapitulando

- Back-end é a cozinha: regra, dado e permissão.
- HTTP é o idioma; verbo, status e JSON são o vocabulário.
- HTTP não tem memória: segure essa ponta até a Aula 3.
- Ruby é Python com outra sintaxe e mais açúcar.
- Rails decide o óbvio por você. Siga a convenção.

Na próxima: o banco de dados entra em cena, e a API ganha um `User` com senha de verdade.